

Assignment 1: Question 1

Course: Machine Learning

Student: Navneet Das

Date: july 2026

Simple Linear Regression — Salary vs. Years of Experience

Objective

Fit a simple linear regression model to predict salary from years of experience.

Step 1: Load the dataset from salary_data.csv.

```
In [1]: import pandas as pd
data = pd.read_csv("./data/Salary_dataset.csv", index_col=0)
data.head()
```

```
Out[1]:
```

	YearsExperience	Salary
0	1.2	39344.0
1	1.4	46206.0
2	1.6	37732.0
3	2.1	43526.0
4	2.3	39892.0

Step 2: Split the data into training and test sets (e.g., 80/20)

```
In [9]: from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(data[["YearsExperience"]], data["Salary"], test_
print("x_train = ", x_train.shape)
print("x_test = ", x_test.shape)
```

```
x_train = (24, 1)
x_test = (6, 1)
```

Step 3: Fit a linear regression model: $y = \beta_0 + \beta_1 x$

```
In [10]: from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
```

```
Out[10]:
```

▼ LinearRegression ⓘ ?

LinearRegression()

Step 4: Display the intercept (β_0) and slope (β_1).

```
In [14]: print("β1 slope of the data",model.coef_[0])
         print("β2 intercept of the data",model.intercept_)
```

```
β1 slope of the data 9423.81532303098
β2 intercept of the data 24380.20147947369
```

Step 5: Predict the salary for a candidate with 7 years of experience.

```
In [18]: model.predict([[7]])
```

```
c:\Users\updes\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

```
Out[18]: array([90346.90874069])
```

Step 6: Evaluate the model using R^2 score and Mean Squared Error (MSE)

```
In [19]: # Generate prediction for the x_test
         y_pred=model.predict(x_test)
         display(y_pred)
```

```
array([115791.21011287,  71499.27809463, 102597.86866063,  75268.80422384,
        55478.79204548,  60190.69970699])
```

```
In [24]: # Calculated metrics
         from sklearn.metrics import mean_squared_error,r2_score

         mse=mean_squared_error(y_test,y_pred)
         r2 = r2_score(y_test,y_pred)

         print(f"Mean squared error = {mse :.4f}")
         print(f"r2 Score = ${r2 :.4f}")
```

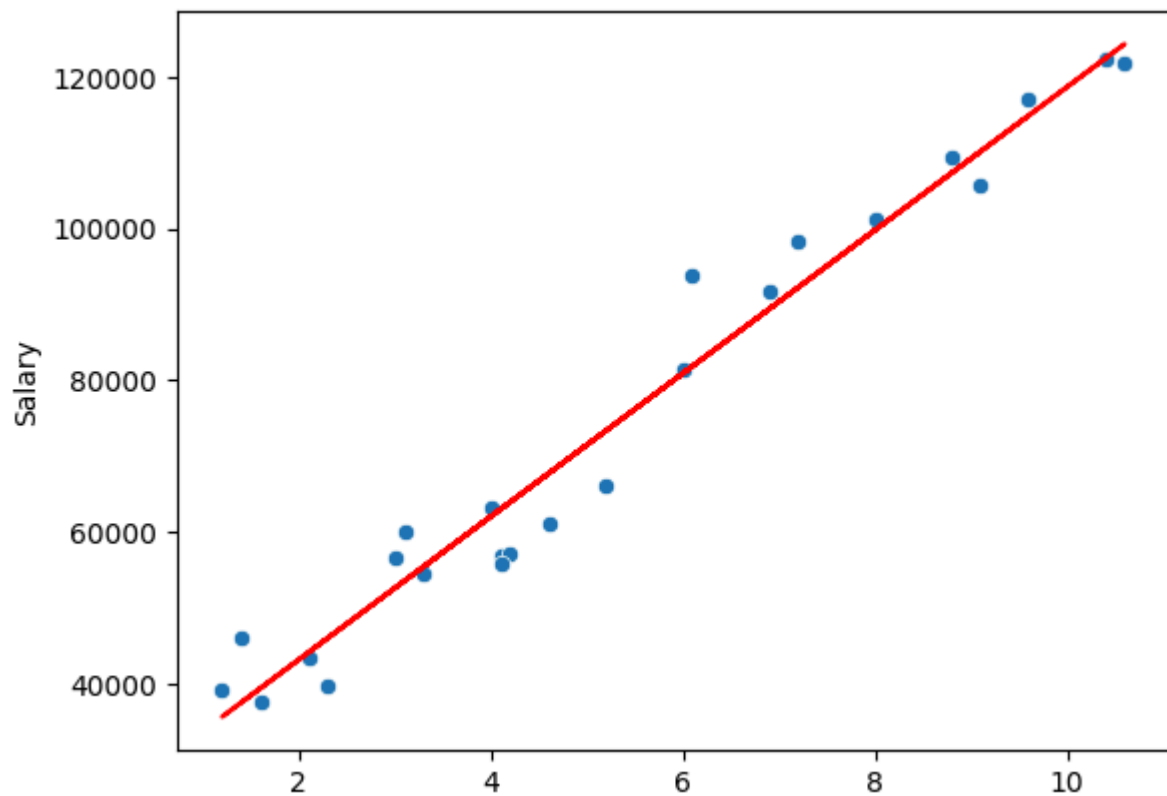
```
Mean squared error = 49830096.8559
r2 Score = $0.9024
```

Step 7: Plot the regression line over the scatter of training data.

```
In [47]: import matplotlib.pyplot as plt
         import seaborn as sns
         import numpy as np

         sns.scatterplot(x=x_train.values.ravel(),y=y_train.squeeze())
         plt.plot(x_train,model.predict(x_train),c="r")
```

```
Out[47]: [<matplotlib.lines.Line2D at 0x1a08e3760f0>]
```



Conclusion

1. **Model Equation:** The simple linear regression model was successfully trained with the equation:

$$\text{Salary} = 24380.20 + 9423.82 \times \text{YearsExperience}$$

- **Slope** ($\beta_1 \approx \$9,423.82$)
- **Intercept** ($\beta_0 \approx \$24,380.20$)

2. **Prediction:**

- The predicted salary for a candidate with **7 years of experience** is approximately **\$90,346.91**.

3. **Model Evaluation:**

- **R^2 Score:** (**\$0.9024\$**)
- **Mean Squared Error (MSE):** **\$49,830,096.86\$**

Assignment 1: Question 2

Course: Machine Learning

Student: Navneet Das

Date: july 2026

Multiple Linear Regression — Predicting House Prices

Objective

Fit a multiple linear regression model to predict house prices based on size and number of bedrooms.

Step 1: Load the dataset from house_prices.csv

```
In [1]: import pandas as pd
data = pd.read_csv("./Data/Housing.csv", usecols=["price", "area", "bedrooms"])
data.sample(5)
```

```
Out[1]:
```

	price	area	bedrooms
374	3640000	3520	2
52	7350000	6000	4
443	3220000	4320	3
261	4473000	3510	3
297	4200000	3640	3

Step 2: Fit a linear regression model: $\text{Price} = \beta_0 + \beta_1(\text{Size}) + \beta_2(\text{Bedrooms})$

```
In [2]: # splitting the data into training and testing data
from sklearn.model_selection import train_test_split
xtrain, xtest, ytrain, ytest = train_test_split(data.iloc[:, [1, 2]], data["price"], test_size=0.2, random_state=42)
print(f"xtrain:{xtrain.shape}\nxtest:{xtest.shape}")
```

```
xtrain:(436, 2)
xtest:(109, 2)
```

```
In [3]: from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(xtrain, ytrain)
```

```
Out[3]:
```

LinearRegression ⓘ ?

LinearRegression()

Step 3: Display the regression coefficients and intercept

```
In [4]: print(f"regression coefficients: {model.coef_}\nregression intercept: {model.intercept_}")
```

regression coefficients: (array([3.89225730e+02, 6.95678667e+05]),)
regression intercept: 642086.4086518651

Step 4: Predict the price for a house with Size = 1800 sq.ft and Bedrooms = 3

```
In [5]: import numpy as np
test=pd.DataFrame({"area":[1800],"bedrooms":[3]})
result = model.predict(test)
print(f"the price for a house with Size = 1800 sq.ft and Bedrooms = 3 will be Rs.{np.round(re
```

the price for a house with Size = 1800 sq.ft and Bedrooms = 3 will be Rs.3429728.72

Step 5: Evaluate the model using R^2 score on a held-out test set

```
In [6]: ypred = model.predict(xtest)

from sklearn.metrics import r2_score
score = r2_score(ytest,ypred)
print(f"The score of the model is {score*100:.3f}%")
```

The score of the model is 35.105%

Step 6: Plot the regression plane in 3D (Size, Bedrooms vs. Price)

```
In [7]: import plotly.graph_objects as go
fig = go.Figure()

# Add scatter trace
fig.add_trace(go.Scatter3d(x=xtest["bedrooms"],y=xtest["area"],z=ytest, mode='markers', name=

# Add line trace
fig.add_trace(go.Scatter3d(x=xtest["bedrooms"],y=xtest["area"],z=ypred, mode='lines', name='L

fig.update_layout(
    width=900,
    height=700,
    title="Resized 3D Overlay Plot"
)

fig.show()
```

Conclusion

1. **Model Equation:** The simple linear regression model was successfully trained with the equation:

$$\text{Price} = 642086.408 + (389.2257 \times \text{area}) + (69567.8667 \times \text{bedrooms})$$

- Slopes ($\beta_1 \approx 3.89225730 \times 10^2$, 6.95678667×10^5)
- Intercept ($\beta_0 \approx 642086.408$)

2. **Prediction:**

- The price for a house with **Size = 1800 sq.ft and Bedrooms = 3** is approximately **\$3429728.72**.

3. **Model Evaluation:**

- **R^2 Score: (0.35105)**

Assignment 1: Question 3

Course: Machine Learning

Student: Navneet Das

Date: july 2026

Polynomial Regression — Modeling Plant Growth Curves

Objective

Fit a polynomial regression model to capture the nonlinear growth pattern of a plant over time.

Alert! The give dataset could not be found heance i have used the **Position_Salaries.csv** dataset in it

step 1: Load the dataset from plant_growth.csv.

```
In [1]: import pandas as pd
data = pd.read_csv("./Data/Position_Salaries.csv", usecols=["Level", "Salary"])
data.shape, data.tail()
```

```
Out[1]: ((10, 2),
         Level  Salary
         5      6   150000
         6      7   200000
         7      8   300000
         8      9   500000
         9     10  1000000)
```

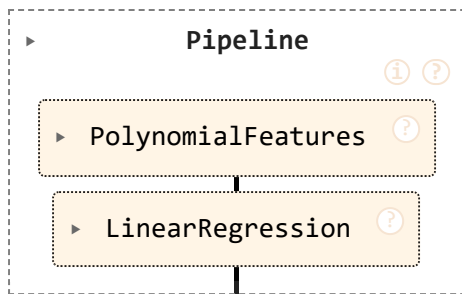
step 2: Fit a degree-3 polynomial: $y = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3$

```
In [2]: xtrain=data[["Level"]]
ytrain=data["Salary"]
```

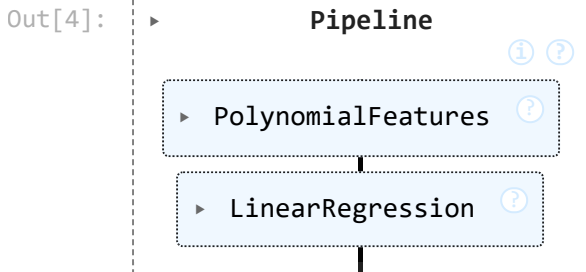
```
In [3]: from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

my_pipeline=make_pipeline(
    PolynomialFeatures(degree=3),
    LinearRegression()
)

display(my_pipeline)
```

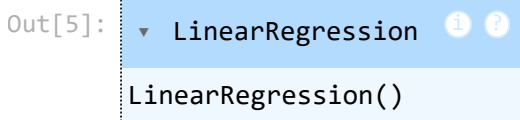


```
In [4]: my_pipeline.fit(xtrain,ytrain)
```



step 3: Also fit a simple linear regression model on the same data for comparison.

```
In [5]: from sklearn.linear_model import LinearRegression
linear_model = LinearRegression()
linear_model.fit(xtrain,ytrain)
```



step 4: Compare the polynomial regression curve with the linear regression line.

```
In [6]: from sklearn.metrics import r2_score
model_r2_score = r2_score(ytrain,my_pipeline.predict(xtrain))
linear_model_r2_score = r2_score(ytrain,linear_model.predict(xtrain))

print(f"the r2 score of polynomial curve is {model_r2_score} or {model_r2_score*100:.2f}%")
print(f"the r2 score of linear line is {linear_model_r2_score} or {linear_model_r2_score*100:.2f}%")
```

the r2 score of polynomial curve is 0.9812097727913365 or 98.12%

the r2 score of linear line is 0.6690412331929895 or 66.90%

step 5: Plot both curves on the same graph, overlaid on the scatter of actual data points.

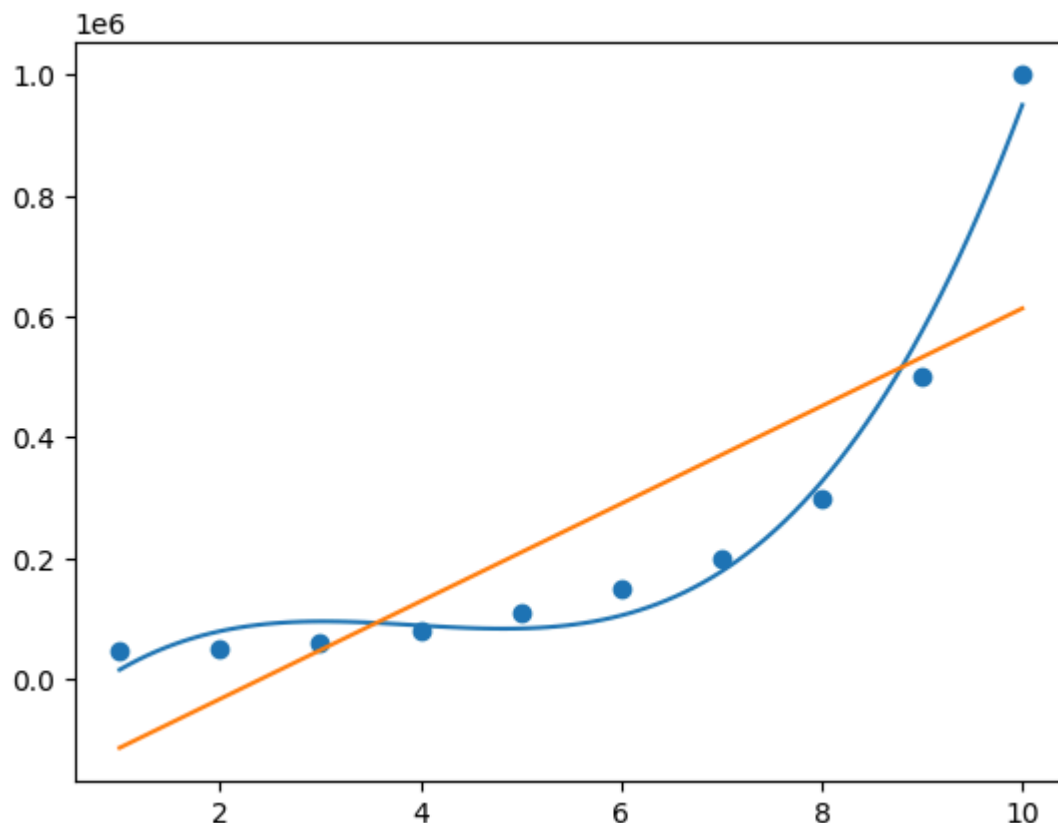
```
In [7]: # numpy.linspace() to generate smooth x-values for plotting the curve
import numpy as np
x_smooth = np.linspace(xtrain.min(),xtrain.max(),100).reshape(-1,1)
y_smooth = my_pipeline.predict(x_smooth)
```

c:\Users\updes\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but PolynomialFeatures was fitted with feature names
warnings.warn(

```
In [8]: import matplotlib.pyplot as plt
import seaborn as sns

plt.scatter(xtrain,ytrain)
plt.plot(x_smooth,y_smooth)
```

```
plt.plot(xtrain,linear_model.predict(xtrain))
plt.show()
```



step 6: Comment on which model fits the data better and why.

Answer: Polynomial regression model fit the curve best with the accuracy of **98.12%** as compaire to Linear Regression model which has a accuracy of only **66.9%**

```
In [18]: # Finding Intercept and coeffecent of the polynomial moded
my_pipeline.named_steps["linearregression"].coef_,my_pipeline.named_steps["linearregression"]
```

```
Out[18]: (array([ 0.          , 180664.33566437, -48548.95104896,  4120.04662005]),
          -121333.33333338297)
```

Conclusion

1. Model Equation:

The simple polynomial regression model was successfully trained with the equation:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

$$\text{Price} = -121333.33 + 0 + (180664.33 \times \text{Level}) + (-48548.95 \times \text{Level}^2) + (4120.04 \times \text{Level}^3)$$

- **Slopes** ($\beta_1 \approx 0$,

$\beta_2 \approx 180664.33566437$, $\beta_3 \approx -48548.95104896$, $\beta_4 \approx 4120.04662005$)

- **Intercept** ($\beta_0 \approx 121333.33333338297$)

2. Model Evaluation:

model	R2 score	-----	-----	Polynomial regression	0.9812097727913365	Linear regression	0.6690412331929895
-------	----------	-------	-------	-----------------------	---------------------------	-------------------	---------------------------

Assignment 1: Question 4

Course: Machine Learning

Student: Navneet Das

Date: july 2026

Polynomial Regression — Choosing the Right Degree (Bias-Variance Tradeoff)

Objective

polynomial regression models of varying degrees to a position-salary dataset and study overfitting/underfitting.

step 1: Load the dataset from position_salaries.csv.

```
In [1]: import pandas as pd
data = pd.read_csv("./Data/Position_Salaries.csv", usecols=["Level", "Salary"])
data.shape, data.tail()
```

```
Out[1]: ((10, 2),
          Level  Salary
5           6   150000
6           7   200000
7           8   300000
8           9   500000
9          10  1000000)
```

step 2: Fit polynomial regression models for degrees 1, 2, 4, and 8.

```
In [2]: from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

trained_models={}
for degree in [1,2,4,8]:
    model_pipeline = make_pipeline(
        PolynomialFeatures(degree=degree),
        LinearRegression()
    )
    model_pipeline.fit(data[["Level"]], data["Salary"])
    print(f"model with degree {degree} trained")
    trained_models[degree]=model_pipeline
    print(f"model saved!")
    print("\n\n", "_"*30)
```

model with degree 1 trained
model saved!

model with degree 2 trained
model saved!

model with degree 4 trained
model saved!

model with degree 8 trained
model saved!

step 3: For each degree, compute the R^2 score / MSE on training data.

```
In [39]: from sklearn.metrics import mean_squared_error, r2_score
mse={}
r2={}
for model in trained_models:
    # print(trained_models[model].named_steps["LinearRegression"].coef_)
    mse[model]=mean_squared_error(data["Salary"],trained_models[model].predict(data[["Level"]])
    r2[model]=r2_score(data["Salary"],trained_models[model].predict(data[["Level"]]))

# displaying the MSE and r2
print("model\t| R2\t\t| mean_squared_error" )
for degree in trained_models:
    print(f"{degree}\t| {r2[degree]:.4f}\t| {mse[degree]:.4f}")
    print(f"{"-"*8}|{"-"*15}|{"-"*20}")
```

model	R2	mean_squared_error
1	0.6690	26695878787.8788
2	0.9162	6758833333.3333
4	0.9974	210343822.8438
8	1.0000	524526.9436

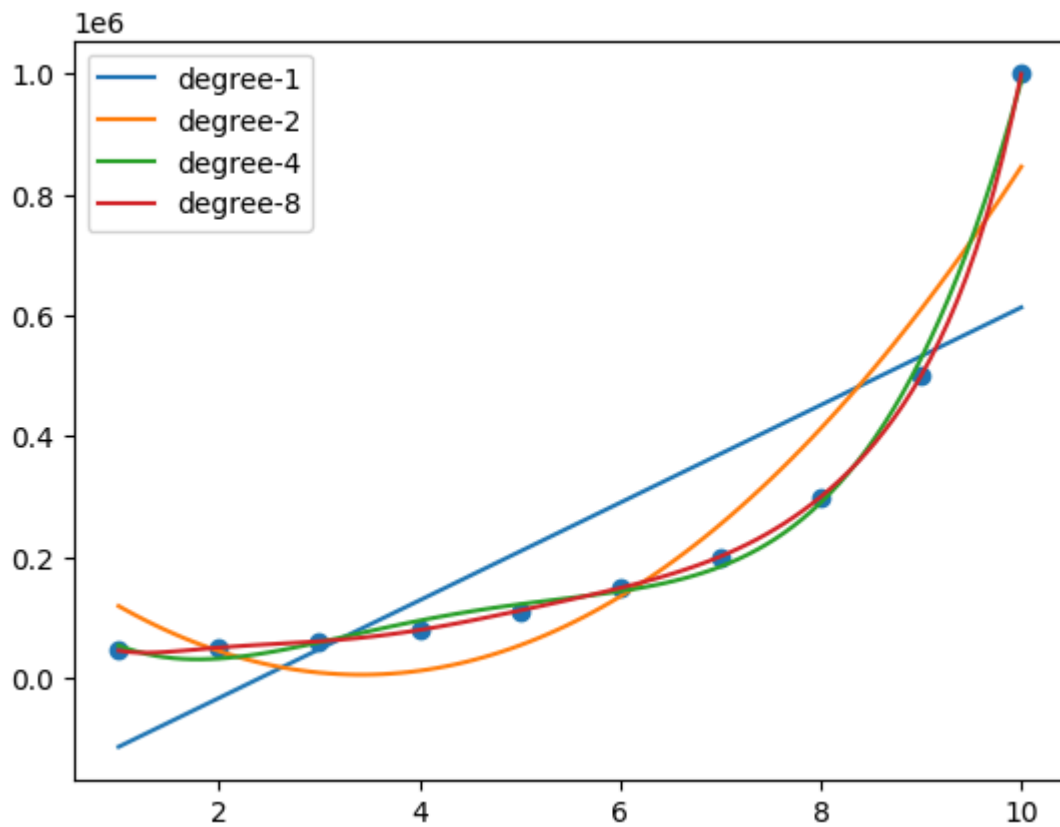
step 4: Plot all fitted curves on the same graph and visually compare underfitting vs. overfitting.

```
In [60]: import warnings
warnings.filterwarnings("ignore")
```

```
In [62]: import matplotlib.pyplot as plt
import numpy as np

x_smooth = np.linspace(data[["Level"]].min(),data[["Level"]].max(),100)

plt.scatter(data["Level"],data["Salary"])
for degree in trained_models:
    y_smooth=trained_models[degree].predict(pd.DataFrame(x_smooth))
    plt.plot(x_smooth,y_smooth,label=f"degree-{degree}")
plt.legend()
plt.show()
```



models with degree	fit
1	underfitting
2	best fitting
4	best fitting
8	overfitting

Heance: we can say that the degree 4 has the best fit

step 5: Predict the salary for Level = 6.5 using the best-performing model.

```
In [71]: predicted_salary = trained_models[4].predict([[6.5]])
print(f"the predicted salary of a person having a experiance of 6.5years will be {predicted_s
```

the predicted salary of a person having a experiance of 6.5years will be 158862.453

step 6: Discuss which degree gives the best trade-off between bias and variance.

Degree 4 gives the best trade-off between bias and variance.

1. Degree 1: High Bias (Underfitting)
2. Degree 2: Moderate Bias
3. Degree 4: Best Trade-off (Optimal Fit)
4. Degree 8: High Variance (Overfitting)